**LAB 7**

**Q1 Implement a queue using Linked list.**

A Node structure/class that contains:

- data → to store the element.
- next → pointer/reference to the next node in the queue.

Two pointers/references:

- front → points to the first node (head of the queue).
- rear → points to the last node (tail of the queue).


Perform the following operations on Queue using Linked List:

1.) Enqueue Operation
2.) Dequeue Operation
3.) isEmpty Operation
4.) Front Operation

Q2 Implement **Priority Queue** using **Linked Lists.** The **Linked List** should be so created so that the **highest priority (** priority is assigned from 0 to n-1 where n is the number of elements, where 0 means the highest priority and n-1 being the least **)** element is always at the **head** of the list. The list is arranged in descending order of elements based on their priority**.** The **Priority Queue** should have the below functions

- **push():** This function is used to insert a new data into the queue.
- **pop():** This function removes the element with the highest priority from the queue.
- **peek() / top():** This function is used to get the highest priority element in the queue without removing it from the queue.

***Example***

***Input:*** *Values = 4 (priority = 1), 5 (priority = 2), 6 (priority = 3), 7 (priority = 0)*
***Output:*** *7 4 5 6*
***Explanation:*** *Values get peeked and popped according to their priority in descending order.*

Q3 You have to implement the **circular queue** with the following operations using a **circular** linked list.

Operations on Circular Queue:

- **Front:** Get the front item from the queue.
- **Rear:** Get the last item from the queue.
- **enQueue(value):** This function is used to **insert** an **element** into the **circular queue**. In a circular queue, the new element is always **inserted** at the **Rear** position.
- **deQueue():** This function is used to **delete** an **element** from the circular queue. In a queue, the element is always **deleted** from the **front** position.